

Lesson 5 Structured Note: Linked-List Stack Fundamentals

0. Current Position in the Course

You are now in **Lesson 5** of the stack curriculum.

This lesson begins:

Phase 2: Linked-List Stack

Before this lesson, you completed **Phase 1: Foundations and Array Stack**.

In Lessons 1–4, you learned:

Lesson	Main idea
Lesson 1	A stack follows LIFO: Last In, First Out.
Lesson 2	An array stack uses <code>size</code> , <code>top</code> , and an array pointer.
Lesson 3	Array stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 4	A complete C array-stack program combines all array-stack functions.

Now Lesson 5 changes the implementation.

The stack rule does **not** change.

The rule is still:

Last In, First Out

The implementation changes from:

Array storage

into:

Linked-list storage

1. Lesson 5 Goal

By the end of this lesson, you should understand:

- What a linked-list stack is.
- What a node is.
- Why each node needs `data` and `next`.
- Why a linked-list stack uses a `top` pointer.
- Why `top` replaces the array stack's `top` index.
- Why push and pop happen at the front of the linked list.
- Why pushing or popping at the end would be inefficient.
- How to write the basic `struct Node`.
- How to initialize an empty linked-list stack.
- How linked-list `push()` works.
- How linked-list `pop()` works.
- What heap overflow means in a linked-list stack.
- What underflow means in a linked-list stack.
- Why linked-list `push()` and `pop()` are $O(1)$.
- How to dry-run push and pop using pointer diagrams.

2. Review: What Is a Stack?

A stack is a linear data structure.

It follows this rule:

LIFO = Last In, First Out

This means:

The last value inserted is the first value removed.

Example:

```
push(10);  
push(20);  
push(30);
```

Logical stack view:

```
Top -> 30  
      20  
      10
```

The last inserted value is **30**.

So if we call:

```
pop();
```

The value removed is:

```
30
```

The stack only allows insertion and deletion at the **top**.

3. Array Stack vs Linked-List Stack

In an array stack, values are stored inside an array.

Example:

Index:	0	1	2
Value:	10	20	30

Here:

```
top = 2
```

So:

```
S[top] = S[2] = 30
```

In an array stack:

```
top is an index
```

A linked-list stack stores values as connected nodes.

Example:

```
top -> 30 -> 20 -> 10 -> NULL
```

In a linked-list stack:

```
top is a pointer
```

Main difference:

Stack type	Meaning of <code>top</code>
Array stack	<code>top</code> is the index of the top element.
Linked-list stack	<code>top</code> is a pointer to the top node.

Important idea:

```
The stack rule stays the same.  
Only the storage method changes.
```

4. What Is a Linked List?

A linked list is a collection of nodes.

Each node stores:

1. A value.
2. A pointer to the next node.

Example:

```
[10 | next] -> [20 | next] -> [30 | NULL]
```

Each box is a node.

The last node points to:

NULL

NULL means:

There is no next node.

5. What Is a Node?

A node is one element of a linked list.

In this lesson, each node stores an integer value.

A node has two parts:

data
next

Node Structure in C

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

Meaning of Each Field

Field	Meaning
data	Stores the actual stack value.
next	Stores the address of the next node.

Example:

```
[ data = 10 | next = NULL ]
```

This node stores the value `10`.

Its `next` pointer is `NULL`, so it does not point to another node.

6. The `top` Pointer

In an array stack, `top` was an integer index.

Example:

```
int top;
```

In a linked-list stack, `top` is a pointer to a node.

Example:

```
struct Node *top;
```

A pointer stores an address.

So:

```
struct Node *top = NULL;
```

means:

```
top currently points to nothing.  
The stack is empty.
```

Example empty stack:

```
top -> NULL
```

Example non-empty stack:

```
top -> 30 -> 20 -> 10 -> NULL
```

Here, `top` points to the node containing `30`.

That node is the top of the stack.

7. Empty Condition in a Linked-List Stack

For an array stack, the empty condition was:

```
top == -1
```

For a linked-list stack, the empty condition is:

```
top == NULL
```

Why?

Because in a linked-list stack, `top` is a pointer.

If `top` points to nothing, there is no node in the stack.

So:

```
top == NULL means the stack is empty.
```

Example:

```
top -> NULL
```

There is no first node.

So the stack has no elements.

8. Why the Front Node Is the Stack Top

In a linked-list stack, we use the **front** of the linked list as the top.

Example:

```
top -> 30 -> 20 -> 10 -> NULL
```

Here:

```
30 is the top element.  
20 is below 30.  
10 is at the bottom.
```

So:

```
pop();
```

removes 30.

And:

```
push(40);
```

adds 40 before 30:

```
top -> 40 -> 30 -> 20 -> 10 -> NULL
```

The front is used because top gives direct access to it.

9. Why We Do Not Push and Pop at the End

Suppose we have this linked-list stack:

```
top -> 30 -> 20 -> 10 -> NULL
```

If we push at the front, we can directly add a new node before 30.

That is fast:

$O(1)$

But if we push at the end, we must first scan the list:

30 -> 20 -> 10 -> NULL

We must move node by node until we reach the last node.

If there are n nodes, this takes:

$O(n)$

That is not good for stack operations.

A stack should perform push and pop at the top in constant time.

So in a linked-list stack:

The front node is used as the top.

10. Array Stack Side vs Linked-List Stack Side

This is a common beginner confusion.

In an array stack, we push and pop at the **right side/end**.

Example:

Index:	0	1	2
Value:	10	20	30
			top

That is efficient because no shifting is required.

In a linked-list stack, we push and pop at the **left side/front**.

Example:

```
top -> 30 -> 20 -> 10 -> NULL
```

That is efficient because `top` directly points to the front node.

Summary:

Implementation	Efficient push/pop side	Reason
Array stack	Right side/end	No shifting needed.
Linked-list stack	Left side/front	<code>top</code> directly points there.

11. Basic Linked-List Stack Setup in C

A simple linked-list stack in C starts like this:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;
```

Why `stdio.h`?

`stdio.h` is used for:

```
printf()
```

Why `stdlib.h`?

`stdlib.h` is used for:

```
malloc()
free()
```

Why `top` Is Global Here

In this beginner C version, `top` is declared outside all functions:

```
struct Node *top = NULL;
```

That makes `top` global.

So functions like `push()` and `pop()` can access it directly.

Important note:

Using a global top pointer is simple for beginners.
Later, a C++ class can hide top inside a class.

12. Memory Used by a Linked-List Stack

A linked-list stack uses heap memory.

Every time we push a value, we create a new node using:

```
malloc()
```

Every time we pop a value, we delete the old top node using:

```
free()
```

So linked-list stack memory grows and shrinks during program execution.

Unlike an array stack, a linked-list stack does not need a fixed size at the start.

Array stack:

```
Fixed capacity after creation.
```

Linked-list stack:

Grows as long as heap memory is available.

13. Overflow in a Linked-List Stack

In an array stack, overflow happens when:

```
top == size - 1
```

That means the array is full.

In a linked-list stack, there is no fixed array size.

So the stack becomes full only if heap memory is not available.

When we try to create a new node:

```
t = (struct Node *)malloc(sizeof(struct Node));
```

If `malloc()` fails, it returns:

```
NULL
```

So linked-list overflow is checked like this:

```
if (t == NULL) {  
    printf("Stack Overflow\n");  
}
```

Simple meaning:

```
Linked-list stack overflow means memory allocation failed.
```

14. Underflow in a Linked-List Stack

Underflow happens when we try to pop from an empty stack.

In a linked-list stack, the stack is empty when:

```
top == NULL
```

So before popping, we check:

```
if (top == NULL) {  
    printf("Stack Underflow\n");  
}
```

Simple meaning:

Underflow means there is no node to remove.

Example:

```
top -> NULL
```

Calling:

```
pop();
```

would cause underflow.

15. Push Operation in a Linked-List Stack

Meaning

`push(x)` inserts a new value at the top of the stack.

Example:

Start:

```
top -> 20 -> 10 -> NULL
```

Call:

```
push(30);
```

Result:

```
top -> 30 -> 20 -> 10 -> NULL
```

The new value `30` becomes the top.

16. Push Code

```
void push(int x) {  
    struct Node *t;  
  
    t = (struct Node *)malloc(sizeof(struct Node));  
  
    if (t == NULL) {  
        printf("Stack Overflow\n");  
    } else {  
        t->data = x;  
        t->next = top;  
        top = t;  
    }  
}
```

17. Push Code Explanation

Step 1: Create a temporary pointer

```
struct Node *t;
```

`t` will point to the new node.

Step 2: Allocate memory for the new node

```
t = (struct Node *)malloc(sizeof(struct Node));
```

This asks the heap for enough memory to store one `struct Node`.

Step 3: Check whether memory allocation failed

```
if (t == NULL)
```

If `t == NULL`, no memory was available.

That means stack overflow.

Step 4: Store the value

```
t->data = x;
```

This puts the value `x` into the new node.

Step 5: Link the new node to the old top

```
t->next = top;
```

This makes the new node point to the previous top node.

Step 6: Move top to the new node

```
top = t;
```

Now the new node becomes the top of the stack.

18. The Most Important Push Pattern

The key push pattern is:

```
t->next = top;  
top = t;
```

Meaning:

1. New node points to the old top.
2. top moves to the new node.

This keeps the linked list connected.

If you update `top` too early, you can lose the old list.

Correct order:

```
t->next = top;  
top = t;
```

Do not reverse this order.

19. Push Dry Run 1: push(10)

Start:

```
top -> NULL
```

Call:

```
push(10);
```

Step 1

Create a new node `t`.

```
t -> [10 | ?]
```

Step 2

Set:

```
t->data = 10;
```

Step 3

Set:

```
t->next = top;
```

Since `top` is currently `NULL`:

```
t -> [10 | NULL]
```

Step 4

Set:

```
top = t;
```

Final result:

```
top -> 10 -> NULL
```

20. Push Dry Run 2: push(15)

Current stack:

```
top -> 10 -> NULL
```

Call:

```
push(15);
```

Create a new node:

```
t -> [15 | ?]
```

Set:

```
t->next = top;
```

The old top points to **10**, so the new node points to **10**:

```
t -> 15 -> 10 -> NULL
```

Set:

```
top = t;
```

Final result:

```
top -> 15 -> 10 -> NULL
```

21. Push Dry Run 3: push(8)

Current stack:

```
top -> 15 -> 10 -> NULL
```

Call:

```
push(8);
```

Create a new node:

```
t -> [8 | ?]
```

Link it to the old top:

```
t->next = top;
```

Now:

```
t -> 8 -> 15 -> 10 -> NULL
```

Move top:

```
top = t;
```

Final result:

```
top -> 8 -> 15 -> 10 -> NULL
```

Logical stack view:

```
Top -> 8  
      15  
      10
```

22. Required Push Dry Run Summary

Operations:

```
push(10);  
push(15);  
push(8);
```

Trace:

Step	Operation	Stack after operation
0	Start	top -> NULL
1	push(10)	top -> 10 -> NULL
2	push(15)	top -> 15 -> 10 -> NULL
3	push(8)	top -> 8 -> 15 -> 10 -> NULL

Final chain:

```
top -> 8 -> 15 -> 10 -> NULL
```

23. Pop Operation in a Linked-List Stack

Meaning

`pop()` removes and returns the top value.

Example:

```
top -> 8 -> 15 -> 10 -> NULL
```

Call:

```
pop();
```

The top value is 8.

So pop() returns:

```
8
```

After popping:

```
top -> 15 -> 10 -> NULL
```

24. Pop Code

```
int pop() {  
    struct Node *p;  
    int x = -1;  
  
    if (top == NULL) {  
        printf("Stack Underflow\n");  
    } else {  
        p = top;  
        x = p->data;  
        top = top->next;  
        free(p);  
    }  
  
    return x;  
}
```

25. Pop Code Explanation

Step 1: Create a temporary pointer

```
struct Node *p;
```

`p` will point to the node that will be deleted.

Step 2: Set default return value

```
int x = -1;
```

If the stack is empty, there is no real value to return.

So `-1` is used as a simple error value.

Step 3: Check underflow

```
if (top == NULL)
```

If `top == NULL`, the stack is empty.

So popping is not possible.

Step 4: Save the old top node

```
p = top;
```

Now `p` points to the node that will be removed.

Step 5: Save the data

```
x = p->data;
```

This saves the value so we can return it later.

Step 6: Move top to the next node

```
top = top->next;
```

Now the second node becomes the new top.

Step 7: Free the old top node

```
free(p);
```

This releases the memory used by the removed node.

Step 8: Return the saved value

```
return x;
```

26. The Most Important Pop Pattern

The key pop pattern is:

```
p = top;  
x = p->data;  
top = top->next;  
free(p);
```

Meaning:

1. Save the old top node.
2. Save its data.
3. Move top to the next node.
4. Free the old top node.

This order is important.

Do not free the top node before moving `top` safely.

27. Pop Dry Run

Start:

```
top -> 8 -> 15 -> 10 -> NULL
```

Call:

```
pop();
```

Step 1: Check if stack is empty

```
top == NULL
```

This is false because `top` points to node `8`.

Step 2: Save old top

```
p = top;
```

Now:

```
p -> 8
```

Step 3: Save data

```
x = p->data;
```

So:

```
x = 8
```

Step 4: Move top

```
top = top->next;
```

Now `top` points to `15`:

```
top -> 15 -> 10 -> NULL
```

Step 5: Free old node

```
free(p);
```

The node containing `8` is removed from heap memory.

Step 6: Return value

```
return x;
```

So `pop()` returns:

```
8
```

Final stack:

```
top -> 15 -> 10 -> NULL
```

28. Required Pop Dry Run Summary

Start:

```
top -> 8 -> 15 -> 10 -> NULL
```

Operation:

```
pop();
```

Trace:

Step	Action	Result
1	<code>p = top</code>	<code>p</code> points to node <code>8</code>
2	<code>x = p->data</code>	<code>x = 8</code>
3	<code>top = top->next</code>	<code>top</code> points to node <code>15</code>
4	<code>free(p)</code>	node <code>8</code> is deleted

Step	Action	Result
5	return x	returns 8

Final stack:

```
top -> 15 -> 10 -> NULL
```

29. Complete Basic Code for Lesson 5

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;

void push(int x) {
    struct Node *t;

    t = (struct Node *)malloc(sizeof(struct Node));

    if (t == NULL) {
        printf("Stack Overflow\n");
    } else {
        t->data = x;
        t->next = top;
        top = t;
    }
}

int pop() {
    struct Node *p;
    int x = -1;

    if (top == NULL) {
        printf("Stack Underflow\n");
    } else {
        p = top;
        x = p->data;
        top = top->next;
    }
}
```

```

        free(p);
    }

    return x;
}

int main() {
    push(10);
    push(15);
    push(8);

    printf("Popped value: %d\n", pop());

    return 0;
}

```

30. Explanation of the Complete Code

Header Files

```

#include <stdio.h>
#include <stdlib.h>

```

`stdio.h` is for input/output.

`stdlib.h` is for heap memory functions.

Node Structure

```

struct Node {
    int data;
    struct Node *next;
};

```

This defines one linked-list node.

Global Top Pointer

```

struct Node *top = NULL;

```

This creates the stack top pointer.

At the beginning, the stack is empty.

Push Calls

```
push(10);  
push(15);  
push(8);
```

After these calls:

```
top -> 8 -> 15 -> 10 -> NULL
```

Pop Call

```
pop();
```

This removes and returns `8`.

After pop:

```
top -> 15 -> 10 -> NULL
```

31. Why `malloc()` Is Used in Push

A linked-list stack does not have an array already waiting with empty slots.

Each new value needs a new node.

So during `push()`, we create one new node using:

```
malloc(sizeof(struct Node))
```

This allocates memory for one node.

Then we fill that node with data and connect it to the stack.

32. Why `free()` Is Used in Pop

When we remove a node, we should release its heap memory.

That is why `pop()` uses:

```
free(p);
```

If we do not free removed nodes, the program wastes memory.

That problem is called a memory leak.

Simple meaning:

```
malloc() creates heap memory.  
free() releases heap memory.
```

33. Array Overflow vs Linked-List Overflow

Array stack overflow:

```
The array is full.
```

Condition:

```
top == size - 1
```

Linked-list stack overflow:

```
The heap cannot provide memory for a new node.
```

Condition:

```
malloc() returns NULL
```

Comparison:

Stack type	Overflow condition
Array stack	<code>top == size - 1</code>
Linked-list stack	<code>malloc()</code> returns <code>NULL</code>

34. Array Underflow vs Linked-List Underflow

Array stack underflow:

Trying to pop when `top == -1`

Linked-list stack underflow:

Trying to pop when `top == NULL`

Comparison:

Stack type	Underflow condition
Array stack	<code>top == -1</code>
Linked-list stack	<code>top == NULL</code>

35. Capacity Difference

An array stack has fixed capacity.

Example:

`size = 5`

It can store only 5 values unless resizing logic is added.

A linked-list stack does not have a fixed capacity at creation.

It grows node by node.

But it is not infinite.

It can still fail if heap memory runs out.

Summary:

Stack type	Capacity behavior
Array stack	Fixed capacity after creation.
Linked-list stack	Grows until heap memory fails.

36. Time Complexity

Push

Linked-list `push()` is $O(1)$.

Why?

It does not scan the list.

It only does these fixed steps:

```
allocate one node
store data
connect new node to old top
move top
```

So:

```
push() =  $O(1)$ 
```

Pop

Linked-list `pop()` is $O(1)$.

Why?

It directly removes the node pointed to by `top`.

It does not scan the list.

So:

```
pop() = O(1)
```

Empty Check

Checking whether the stack is empty is $O(1)$.

Condition:

```
top == NULL
```

Display

Display is not the main focus of Lesson 5, but if we print all nodes, we must traverse the whole list.

So display is:

```
O(n)
```

37. Complexity Summary Table

Operation	Time complexity	Auxiliary space	Reason
<code>push()</code>	$O(1)$	$O(1)$	Adds one node at the front.
<code>pop()</code>	$O(1)$	$O(1)$	Removes one node from the front.
Empty check	$O(1)$	$O(1)$	Checks whether <code>top == NULL</code> .
Display	$O(n)$	$O(1)$	Traverses all nodes.

Important point:

```
Linked-list stack push and pop are  $O(1)$  because they happen at the front.
```

38. Space Complexity

Each node stores:

```
data
next pointer
```

If the stack has `n` values, it has `n` nodes.

So total stack storage is:

```
O(n)
```

But one individual `push()` operation uses only a fixed number of extra variables.

So auxiliary space per operation is:

```
O(1)
```

39. Full Comparison: Array Stack vs Linked-List Stack

Concept	Array stack	Linked-list stack
Storage	Array	Nodes
Top representation	Integer index	Pointer to node
Empty condition	<code>top == -1</code>	<code>top == NULL</code>
Full condition	<code>top == size - 1</code>	<code>malloc()</code> returns <code>NULL</code>
Push location	Right/end of array	Front of linked list
Pop location	Right/end of array	Front of linked list
Need fixed size?	Yes	No
Memory allocation	One array allocation	One node allocation per push
Memory release	Free array at end	Free node after each pop
Push time	$O(1)$	$O(1)$
Pop time	$O(1)$	$O(1)$
Can grow?	Not automatically	Yes, until heap fails

40. Important Pointer Diagrams

Empty Stack

```
top -> NULL
```

After push(10)

```
top -> 10 -> NULL
```

After push(15)

```
top -> 15 -> 10 -> NULL
```

After push(8)

```
top -> 8 -> 15 -> 10 -> NULL
```

After pop()

```
top -> 15 -> 10 -> NULL
```

Popped value:

```
8
```

41. Common Beginner Mistakes

Mistake 1: Thinking `top` is still an index

In an array stack:

```
top is an index
```

In a linked-list stack:

```
top is a pointer
```

Do not confuse them.

Mistake 2: Using `top == -1` for a linked-list stack

Wrong for linked list:

```
top == -1
```

Correct:

```
top == NULL
```

Because `top` is a pointer, not an integer index.

Mistake 3: Trying to push at the end of the linked list

Wrong idea:

```
Insert after the last node.
```

Problem:

```
You must scan the whole list.
```

That becomes:

```
O(n)
```

Correct idea:

```
Push at the front where top already points.
```

That stays:

0(1)

Mistake 4: Reversing the push links

Correct push order:

```
t->next = top;  
top = t;
```

Wrong order:

```
top = t;  
t->next = top;
```

The wrong order can lose the old stack connection or create incorrect links.

Mistake 5: Forgetting to check `malloc()`

Wrong:

```
t = (struct Node *)malloc(sizeof(struct Node));  
t->data = x;
```

Problem:

If `malloc()` fails, `t` is `NULL`.

Using `t->data` would be unsafe.

Correct:

```
t = (struct Node *)malloc(sizeof(struct Node));  
if (t == NULL) {  
    printf("Stack Overflow\n");  
}
```

Mistake 6: Forgetting underflow check before pop

Wrong:

```
p = top;  
x = p->data;
```

Problem:

If `top == NULL`, then `p` is also `NULL`.

Accessing `p->data` would be invalid.

Correct:

```
if (top == NULL) {  
    printf("Stack Underflow\n");  
}
```

Mistake 7: Freeing the node too early

Wrong idea:

```
free(top);  
top = top->next;
```

Problem:

After freeing `top`, using `top->next` is unsafe.

Correct order:

```
p = top;  
x = p->data;  
top = top->next;  
free(p);
```

Mistake 8: Forgetting to free popped nodes

If you remove a node but do not call `free()`, the memory remains allocated.

That causes a memory leak.

Correct:

```
free(p);
```

42. Key Rules and Formulas

Node Structure

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

Empty Condition

```
top == NULL
```

Push Pattern

```
t = malloc(sizeof(struct Node));  
t->data = x;  
t->next = top;  
top = t;
```

Pop Pattern

```
p = top;  
x = p->data;  
top = top->next;  
free(p);  
return x;
```

Overflow Condition

```
malloc() returns NULL
```

Underflow Condition

```
top == NULL
```

Front-Side Rule

Push and pop at the front because top directly points there.

43. Required Practice 1: Draw the Stack

Start:

```
top -> NULL
```

Operations:

```
push(4);  
push(9);  
push(2);
```

Step-by-step:

After `push(4)`:

```
top -> 4 -> NULL
```

After `push(9)`:

```
top -> 9 -> 4 -> NULL
```

After `push(2)`:

```
top -> 2 -> 9 -> 4 -> NULL
```

Final answer:

```
top -> 2 -> 9 -> 4 -> NULL
```

44. Required Practice 2: Dry-Run One Pop

Start:

```
top -> 2 -> 9 -> 4 -> NULL
```

Call:

```
pop();
```

The top value is .

So:

```
pop returns 2
```

After pop:

```
top -> 9 -> 4 -> NULL
```

The node containing is freed.

45. Required Practice 3: Compare Full Conditions

Array stack full condition:

```
top == size - 1
```

Linked-list stack full condition:

```
malloc() returns NULL
```

Simple explanation:

Array stack is full when the array has no free slot.
Linked-list stack is full when the heap cannot create a new node.

46. Mini Trace Practice

Trace this sequence:

```
push(5);  
push(12);  
push(7);  
pop();  
push(20);  
pop();
```

Step-by-step answer:

Step	Operation	Return value	Stack after operation
0	Start	—	top -> NULL
1	push(5)	—	top -> 5 -> NULL
2	push(12)	—	top -> 12 -> 5 -> NULL
3	push(7)	—	top -> 7 -> 12 -> 5 -> NULL
4	pop()	7	top -> 12 -> 5 -> NULL
5	push(20)	—	top -> 20 -> 12 -> 5 -> NULL
6	pop()	20	top -> 12 -> 5 -> NULL

Final stack:

```
top -> 12 -> 5 -> NULL
```

47. Checkpoint Questions

Try to answer these without looking at the answers.

1. What is a linked-list stack?
 2. What are the two fields inside a node?
 3. What does `data` store?
 4. What does `next` store?
 5. What does `top` mean in a linked-list stack?
 6. What is the empty condition in a linked-list stack?
 7. Why do we push at the front of the linked list?
 8. Why do we not push at the end of the linked list?
 9. What does `malloc()` do during push?
 10. What does `free()` do during pop?
 11. What causes overflow in a linked-list stack?
 12. What causes underflow in a linked-list stack?
 13. What is the key push pattern?
 14. What is the key pop pattern?
 15. What is the time complexity of linked-list `push()` ?
 16. What is the time complexity of linked-list `pop()` ?
 17. How is array stack overflow different from linked-list stack overflow?
 18. How is array stack `top` different from linked-list stack `top` ?
-

48. Checkpoint Answers

1. A linked-list stack is a stack implemented using linked-list nodes.
 2. A node has `data` and `next` .
 3. `data` stores the actual value.
 4. `next` stores the address of the next node.
 5. `top` points to the first node, which is the stack top.
 6. The empty condition is `top == NULL` .
 7. We push at the front because `top` directly points there, so insertion is $O(1)$.
 8. We do not push at the end because reaching the end requires traversal, which is $O(n)$.
 9. `malloc()` creates memory for a new node.
 10. `free()` releases memory of a removed node.
 11. Linked-list overflow happens when `malloc()` returns `NULL` .
 12. Underflow happens when we pop while `top == NULL` .
 13. The key push pattern is `t->next = top; top = t; .`
 14. The key pop pattern is `p = top; x = p->data; top = top->next; free(p); .`
 15. Linked-list `push()` is $O(1)$.
 16. Linked-list `pop()` is $O(1)$.
 17. Array overflow means the array is full; linked-list overflow means heap allocation failed.
 18. Array stack `top` is an index; linked-list stack `top` is a pointer.
-

49. Short Glossary

Term	Simple meaning
Stack	A data structure that follows LIFO.
LIFO	Last In, First Out.
Linked list	A collection of connected nodes.
Node	A structure containing data and a pointer to the next node.
<code>data</code>	The value stored inside a node.
<code>next</code>	Pointer to the next node.
<code>top</code>	Pointer to the first node in a linked-list stack.
<code>NULL</code>	Pointer value meaning "points to nothing."
<code>malloc()</code>	C function used to allocate heap memory.
<code>free()</code>	C function used to release heap memory.
Push	Insert a value at the top.
Pop	Remove and return the top value.
Overflow	Trying to push when memory is not available.
Underflow	Trying to pop when the stack is empty.
Heap	Memory area used for dynamic allocation.
$O(1)$	Constant time. Work does not grow with the number of nodes.
$O(n)$	Linear time. Work grows with the number of nodes.

50. Final Lesson 5 Summary

Lesson 5 teaches how to implement a stack using a linked list.

The stack rule remains:

Last In, First Out

The implementation changes.

In an array stack:

top is an index.

In a linked-list stack:

top is a pointer to the first node.

A node has two parts:

```
int data;  
struct Node *next;
```

An empty linked-list stack is represented as:

top == NULL

Push inserts a new node at the front:

```
t->data = x;  
t->next = top;  
top = t;
```

Pop removes the front node:

```
p = top;  
x = p->data;  
top = top->next;  
free(p);
```

A linked-list stack uses the front because `top` directly reaches the first node.

That makes both push and pop:

$O(1)$

The main idea to remember is:

Array stack controls top as an index.
Linked-list stack controls top as a pointer.
The LIFO rule stays the same.

If you can draw pointer changes for `push()` and `pop()`, you understand the core of Lesson 5.